

Direct3D 12: da aplicacao ao pixel no monitor

Pipeline grafico, comunicacao CPU/GPU, tessellation e iluminacao

Do codigo ao monitor



Trabalho academico - TT1 - C09C

DirectX / Direct3D 12 - demonstracao pratica em C++ e HLSL
2026

1. Objetivo do trabalho

Este trabalho explica como uma aplicacao consegue produzir uma imagem usando Direct3D 12 e como essa imagem percorre o sistema ate chegar fisicamente ao monitor. O cubo interativo e a superficie Bezier sao demonstracoes visuais; o foco principal e o funcionamento da API, a divisao de responsabilidades entre CPU e GPU e o pipeline de renderizacao.

- Demonstrar a comunicacao entre aplicacao, Direct3D, driver e GPU.
- Explicar como vertices e comandos se transformam em pixels.
- Mostrar sincronizacao CPU/GPU com Command Queue, Command List e Fence.
- Demonstrar transformacoes, cores, superficies curvas, tessellation e iluminacao.
- Explicar Swap Chain, Present, DWM, scanout e a chegada do frame ao monitor.

2. O que e DirectX e o que e uma API?

DirectX nao e um unico programa. E uma familia de APIs e componentes do Windows voltados a multimidia, principalmente graficos, audio e integracao com hardware. Neste projeto, o componente central e o Direct3D 12, acompanhado por DXGI e HLSL.

API significa Application Programming Interface. Uma API e um contrato de software: define funcoes, interfaces, estruturas de dados, parametros e regras que uma aplicacao usa para solicitar operacoes sem precisar conhecer a implementacao interna do hardware.

Componente	Funcao no projeto
Direct3D 12	Recursos, pipeline, comandos e sincronizacao para a GPU
DXGI	Adapters, formatos, Swap Chain e apresentacao
HLSL	Linguagem dos shaders executados na GPU
DirectXMath	Matrizes e vetores usados pela aplicacao

Exemplos reais de chamadas de API usadas no projeto: D3D12CreateDevice cria a interface logica com o adapter; CreateCommandQueue cria uma fila de trabalho; ExecuteCommandLists submete comandos; Present solicita a apresentacao de um back buffer.

3. Software, driver e hardware

O Direct3D fornece uma interface padronizada, mas GPUs diferentes possuem arquiteturas internas diferentes. O driver do fabricante faz a ponte entre a API e o hardware. Por isso o mesmo programa pode usar GPUs NVIDIA, AMD ou Intel sem conhecer suas instruções privadas.

Do código ao monitor



DirectX pode ser simplesmente atualizado?

Somente em parte. No Windows moderno, componentes centrais do DirectX fazem parte do sistema operacional e são atualizados pelo Windows Update. O driver gráfico é atualizado separadamente. Porém, atualizar software não cria capacidades físicas que a GPU não possui.

API, Feature Level e Shader Model

Conceito	Exemplo do projeto	Significado
API	Direct3D 12	Modelo de programação usado pela aplicação
Feature Level	11_0	Capacidade mínima exigida do hardware/driver
Shader Model	5.1	Perfil usado para compilar os shaders HLSL

Portanto, usar Direct3D 12 com D3D_FEATURE_LEVEL_11_0 não significa usar Direct3D 11. A versão da API e o nível de recursos são eixos diferentes.

4. CPU e GPU: responsabilidades diferentes

A CPU e a GPU são processadores distintos. A CPU executa a lógica do programa, processa o mouse, calcula matrizes e grava comandos. A GPU recebe esses comandos e executa milhares de operações gráficas em paralelo.

CPU e GPU trabalham de forma assíncrona



CPU	GPU
Processa Win32 e input	Executa shaders
Atualiza camera e matrizes	Transforma vertices
Grava Command Lists	Rasteriza triangulos
Gerencia recursos e estados	Calcula cores e iluminacao
Coordena com Fences	Escreve no back buffer

ExecuteCommandLists submete trabalho para a GPU, mas nao espera que ele termine. A aplicacao usa Fences para saber quando e seguro reutilizar Command Allocators e Constant Buffers de um frame anterior.

5. Como um frame e gerado

Pipeline basico do cubo



No cubo, o Input Assembler le o Vertex Buffer e o Index Buffer. O Vertex Shader transforma os vertices. A rasterizacao determina quais amostras da tela sao cobertas pelos triangulos. O Pixel Shader calcula a cor. O Output Merger aplica o teste de profundidade e escreve no Render Target, que neste projeto e o back buffer atual da Swap Chain.

Objetos importantes do Direct3D 12

- ID3D12Device: interface logica para criar objetos D3D12.
- Command List: sequencia de operacoes gravadas pela CPU.
- Command Allocator: memoria usada pela gravacao da lista.
- Command Queue: fila que recebe as listas e as envia para execucao.
- Pipeline State Object (PSO): agrupa shaders e estados fixos.
- Root Signature: contrato de recursos acessiveis aos shaders.
- Resource Barrier: declara transicoes de estado dos recursos.
- Fence: marcador de progresso usado para sincronizar CPU e GPU.

6. Transformacoes e camera

Um objeto 3D nao chega pronto a GPU. O cubo e definido por numeros: posicoes, cores, normais e indices. A CPU calcula tres matrizes principais: Model, View e Projection.

Matriz	Papel
Model	Posiciona e orienta o objeto no mundo
View	Representa a camera e transforma o mundo

	para o espaço de visao
Projection	Aplica perspectiva e prepara a cena para a tela

No projeto: $MVP = Model * View * Projection$. A matriz é enviada por Constant Buffer ao Vertex Shader, que calcula a nova posição de cada vértice.

Do movimento do mouse a imagem



Quando o usuário arrasta o mouse, o Windows gera WM_MOUSEMOVE. A CPU atualiza yaw e pitch da câmera, recalcula a View Matrix, envia a nova MVP e a GPU produz outro frame. A CPU não envia pixels diretamente ao monitor.

7. Curvas e superfícies: tessellation da GPU

Para demonstrar curvas e superfícies, o projeto implementa uma superfície Bezier bicúbica definida por uma grade 4x4 de 16 pontos de controle. A CPU envia apenas esses pontos e um fator de tessellation; a subdivisão ocorre no pipeline da GPU.

Pipeline da superfície Bezier



Estágios adicionais

- Hull Shader: define fatores de tessellation nas bordas e no interior do patch.
- Tessellator: unidade fixed-function que gera coordenadas paramétricas (u, v).
- Domain Shader: avalia a superfície Bezier usando bases de Bernstein e calcula a normal por derivadas dP/du e dP/dv .

O fator pode variar de 1 a 32. Em wireframe, fator 1 mostra poucos triângulos; fatores maiores produzem uma malha progressivamente mais densa. Isso permite demonstrar nível de detalhe sem enviar uma malha altamente subdividida pela CPU.

8. Iluminacao e realismo basico

O Direct3D nao cria luzes automaticamente. A aplicacao define a equacao de iluminacao e os shaders executam os calculos. O projeto usa um modelo Blinn-Phong didatico:

$$I = \text{ambiente} + \text{difusa} + \text{especular}$$

- Ambiente: contribuicao minima de luz, independente da orientacao.
- Difusa: depende do produto escalar entre normal e direcao da luz.
- Especular: usa normal, direcao da luz e posicao da camera para produzir brilho.

O cubo possui normais planas por face. Na superficie Bezier, as normais sao calculadas no Domain Shader a partir das derivadas da superficie. O painel permite comparar iluminacao desligada, iluminacao ligada e componente especular.

Esse resultado representa realismo basico, nao fotorrealismo. Tecnicas mais avancadas poderiam incluir texturas, sombras, PBR, normal mapping, reflexos, ambient occlusion e ray tracing.

9. Do back buffer ao monitor

A GPU escreve o frame em um Render Target. Neste projeto, esse recurso e um dos tres back buffers da Swap Chain. Antes de desenhar, uma Resource Barrier muda o estado PRESENT para RENDER_TARGET. Ao terminar, outra barrier retorna para PRESENT.

Present(1, 0) solicita a apresentacao sincronizada com o intervalo vertical. Em uma janela normal, o Windows/DWM pode compor o frame com outras janelas. Depois, o display engine da GPU realiza scanout, envia o sinal pela saida de video e o controlador do monitor atualiza o painel.

A cadeia conceitual completa e:

Aplicacao -> CPU -> Direct3D/DXGI -> Driver -> GPU -> Back Buffer -> Present -> DWM -> Display Engine -> Monitor

Triple buffering

A Swap Chain possui tres back buffers. Isso permite sobreposicao entre preparacao de frames, execucao da GPU e apresentacao. Tres buffers nao significam obrigatoriamente tres frames executando shaders ao mesmo tempo; representam tres contextos que podem estar em fases diferentes do fluxo.

10. Demonstracao pratica

A aplicacao foi criada em C++20, Win32, Direct3D 12, HLSL e DirectXMath. A interface Dear ImGui e desenhada na mesma Command List da cena e permite alterar parametros em tempo real.

Controle	Acao
Mouse esquerdo + arrastar	Orbitar a camera
Scroll	Zoom
R	Reset da camera

Espaco	Auto-rotacao
1	Cubo
2	Superficie Bezier
W	Wireframe
L	Iluminacao
C	Visualizacao por posicao
F1	Mostrar/ocultar interface
Esc	Sair

Pelo painel tambem e possivel alterar fator de tessellation, direcao e intensidade da luz, ambiente, componente especular e shininess. A telemetria mostra FPS, GPU selecionada, indice do back buffer, Fence Value, yaw, pitch e zoom.

11. Conclusao

Direct3D 12 e uma API grafica de baixo nivel relativo: ela nao fornece objetos, camera ou iluminacao prontos, mas oferece os mecanismos necessarios para que a aplicacao controle recursos, pipeline, comandos e sincronizacao. O projeto demonstra esse caminho de forma concreta, do movimento do mouse ate o novo frame no monitor, incluindo tessellation de uma superficie curva e iluminacao programavel.

A principal ideia e que renderizar e apresentar sao etapas diferentes: a CPU descreve e coordena, a GPU produz a imagem em memoria e o sistema de apresentacao leva essa imagem ao display.

Referencias

1. Microsoft Learn - What is DirectX 12?.

<https://learn.microsoft.com/windows/win32/direct3d12/what-is-directx-12->

2. Microsoft Learn - Hardware Feature Levels.

<https://learn.microsoft.com/windows/win32/direct3d12/hardware-feature-levels>

3. Microsoft Learn - Command Queues and Command Lists.

<https://learn.microsoft.com/windows/win32/direct3d12/command-queues-and-command-lists>

4. Microsoft Learn - Pipelines and Shaders with DirectX 12.

<https://learn.microsoft.com/windows/win32/direct3d12/pipelines-and-shaders-with-directx-12>

5. Microsoft Learn - Tessellation stages.

<https://learn.microsoft.com/windows/win32/direct3d11/direct3d-11-advanced-stages-tessellation>

6. Microsoft Learn - DXGI flip model.

<https://learn.microsoft.com/windows/win32/direct3ddxgi/dxgi-1-2-presentation-improvements>

7. Microsoft Learn - IDXGISwapChain::Present.

<https://learn.microsoft.com/windows/win32/api/dxgi/nf-dxgi-idxgiswapchain-present>

8. Microsoft Learn - WARP. <https://learn.microsoft.com/windows/win32/direct3darticles/directx-warp>

9. Microsoft DirectX Graphics Samples. <https://github.com/microsoft/DirectX-Graphics-Samples>